CropCare AI: Field-Reliable Crop Disease Detection

Final-year project report · Oct 7, 2026 · @Mohammad Aftab

Abstract

Crop disease classifiers trained on lab photos look excellent on paper and fail in the field. We measured this directly: a MobileNetV3 model trained only on PlantVillage scored 99.1% on lab test photos but 27.3% on real field photos (PlantDoc), a 72-point drop.

CropCare AI is built around closing and honestly reporting that gap. Training on lab and field photos together (PlantVillage, PlantDoc, PlantWild, rice and wheat field sets) with source-balanced sampling and near-duplicate removal raised field accuracy to 69.3% across 541 field test photos, while keeping 98.9% on lab photos. The deployed model (Experiment D) covers 30 classes, including four insect-damage classes, across six crops: pepper, potato, tomato, maize, rice and wheat.

Instead of always naming a disease, the system says when it is unsure. Temperature scaling and conformal prediction give every answer one of four statuses: confident, ambiguous ("could be A or B"), unknown, or rejected (not a supported crop leaf). On field photos the true disease falls inside the model's answer set 90.0% of the time, matching the 90% target, and confident answers are correct 87.4% of the time.

The application adds lesion-based severity (U-Net, IoU 0.63), Grad-CAM explanations, multi-photo diagnosis, a three-day weather-based disease risk forecast, safety-first treatment advice, Hindi advice translated on the project's own server (NLLB-200 model with a farming glossary and reviewed corrections, no paid API), and an offline mode. It is built with React, Spring Boot with MySQL, and a FastAPI/PyTorch AI service, and is covered by 70 automated tests and end-to-end browser tests.

1. Introduction and problem statement

Plant diseases cause large yield losses, and smallholder farmers often cannot reach an expert in time. A phone camera and a classifier promise a first diagnosis in seconds, which is why crop disease detection is one of the most common AI student projects.

Most of these projects share one recipe: a CNN trained on the PlantVillage dataset, reporting 97–99% accuracy. PlantVillage photos are taken in a lab, one leaf on a plain background under even light. Published studies show models trained this way collapse on

field photos, and that PlantVillage carries background cues a model can learn instead of the disease.

We confirmed the problem on our own pipeline before building anything else:

Model trained on	Lab test (PlantVillage)	Field test (PlantDoc)
Lab photos only (MobileNetV3)	99.1%	27.3%

A 99% lab score therefore says little about how the app behaves for a farmer. A second, quieter problem is that a standard classifier always names a disease, even for a blurry photo, an unsupported crop or a picture of a hand. In agriculture a confident wrong answer leads to the wrong spray, wasted money and crop damage.

Problem statement. Build a crop disease detection system whose accuracy is measured on real field photos, which says when it does not know, and which turns a diagnosis into safe, local, understandable action for Indian farmers.

2. Literature review

Prior work agrees on the lab-to-field gap but rarely ships a system that measures it or refuses to guess.

Theme	What the literature reports	How this project uses it
Lab-to-field gap	A ResNet-50 trained on PlantVillage scored 99.73% on PlantVillage and 32.05% on PlantDoc; domain adaptation only reached about 36% (Frontiers in Plant Science, 2026)	Experiment A reproduces this gap; Experiment B and C close part of it with field data
Dataset bias	A model using only 8 background pixels of PlantVillage reached 49% accuracy against 2.6% chance (Noyan, 2022)	Heavy augmentation, field datasets, and evaluation on field test sets only
In-the-wild datasets	PlantDoc and PlantWild (18,542 web photos, 89 classes) give field-like photos (PlantWild, ACM MM 2024); PlantSeg gives 11,400 lesion masks (PlantSeg)	PlantDoc and PlantWild for training and testing, PlantSeg for severity

Theme	What the literature reports	How this project uses it
Foundation models	DINOv2 features generalised best in the 2026 study (43% vs 32% on PlantDoc); BioCLIP is trained on the tree of life (BioCLIP 2)	DINOv2-Small compared as a backbone; BioCLIP tested as a plant gate
Uncertainty	Conformal prediction gives answer sets with guaranteed coverage and supports abstaining (Computers and Electronics in Agriculture, 2025)	Conformal prediction decides the four statuses
Severity	Segmentation networks estimate the diseased leaf area for severity (IoU 85.9%, R^2 0.908 on three diseases) (PDSNets)	U-Net lesion model trained on PlantSeg
Vision-language models	General VLMs exceed 90% on healthy vs diseased but stay below 65% on fine-grained disease names (LeafNet benchmark)	A trained classifier decides; the LLM only answers follow-up questions about that decision
Existing apps	Reviewed apps lack offline use, local languages, multi-disease handling and concrete treatment advice (Evaluating plant disease apps); Plantix is the main Indian example (CGIAR case study)	Offline mode, Hindi, multi-photo diagnosis and structured advice

3. Objectives and novelty

The project's contribution is not a new network architecture. It is an honestly evaluated, safety-first system: every number is measured on field photos, and the app refuses to treat what it cannot identify.

Objectives

1. Measure the lab-to-field accuracy gap on our own pipeline and report accuracy separately for lab and field test sets.
2. Reduce that gap with field data, augmentation and sampling, and compare four backbones on the same data.

3. Make every prediction say how sure it is, with a verified coverage guarantee, and reject photos that are not supported crop leaves.
4. Estimate disease severity and explain each prediction visually.
5. Turn a diagnosis into safe, local action: weather risk, treatment advice that never recommends spraying for an unconfirmed disease, and Hindi.
6. Work with poor connectivity: installable app with on-device diagnosis when offline.

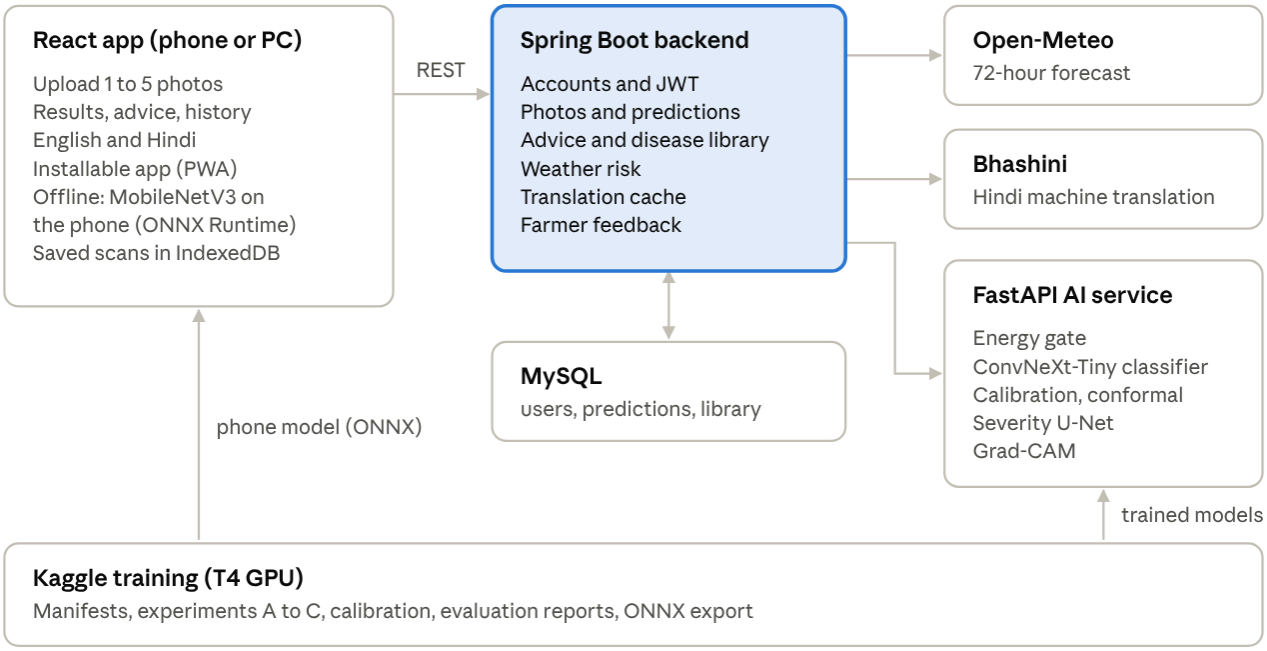
What is new compared with typical projects

- Field-first evaluation: lab and field accuracy reported side by side, with leakage and near-duplicate checks between training and test photos.
- Four honest statuses instead of a single label, from calibrated probabilities and conformal prediction.
- A measured negative result: the zero-shot BioCLIP gate rejected about 8% of genuine field leaves, so it is not used by default.
- Severity, Grad-CAM, weather risk, Hindi advice and offline diagnosis combined in one working system.

4. System architecture

The browser talks only to the Spring Boot backend, which owns all data and calls the AI service and Open-Meteo; the AI service classifies and translates advice into Hindi.

The phone talks only to the backend; the AI service only classifies



system architecture · 3 services, 3 external parts

Models are trained on Kaggle and copied in: the server model into the AI service, the phone model into the web app for offline use.

5. Datasets and data pipeline

Five public datasets were mapped onto one list of 26 classes, and 1,650 training photos were removed because they were copies or near-copies of validation or test photos.

Dataset	Kind of photo	Images used	Train / val / test	Exact duplicates removed
PlantVillage (pepper, potato, tomato, maize)	Lab, plain background	26,625	21,302 / 2,661 / 2,662	14
PlantDoc	Field, web	1,465	1,070 / 267 / 128	11
PlantWild (19 of 89 classes)	Field, web	4,147	3,317 / 417 / 413	125
Rice leaf disease (Sethy)	Field	3,486	2,789 / 349 / 348	1,138
Wheat disease (Kaggle)	Field	7,763	6,640 / 1,123 / none shipped	2,906
PlantSeg (severity only)	Field, lesion masks	9,163	7,916 / 1,247	n/a

One class list. Every dataset names its folders differently ("Tomato___Late_blight", "Tomato leaf late blight", "tomato late blight"). A taxonomy file (`configs/taxonomy.yaml`) lists the 26 canonical classes with aliases; a folder is matched after removing case and punctuation, and with a crop prefix for crop-only datasets such as rice ("Blast" becomes rice leaf blast). Folders outside the taxonomy, such as apple or tungro, are skipped and listed.

Manifests, not copies. Each dataset becomes a CSV manifest of path, class, source, split, SHA-1 and a 64-bit perceptual hash (dHash). Images are never copied, so the same code runs on a laptop and on Kaggle's read-only storage. Random splits are stratified 80/10/10 per class with a fixed seed; PlantDoc keeps its own train/test split, with 20% of train moved to validation.

Leakage control. Exact duplicates are removed inside each dataset. Before training, every training photo is compared with every validation and test photo; any photo with the same SHA-1 or a dHash within 4 bits is dropped from training. This removed 432 PlantVillage, 58

PlantDoc, 115 PlantWild, 800 rice and 245 wheat photos. Web-scraped sets share re-saved copies of the same photo, so without this check field accuracy would be inflated.

Known data issue. The rice dataset contains rotated and flipped copies of the same photos, which dHash does not catch. Its 100% test accuracy is therefore not reported as a result.

6. Model development

Three experiments isolate what moves field accuracy: lab-only data (A), adding PlantDoc field photos (B), and adding PlantWild, rice and wheat with source-balanced sampling (C).

Experiment	Training data	Backbone	Classes	Image size	Training photos
A: lab only	PlantVillage	MobileNetV3-Large	19	224	21,302
B: lab + field	PlantVillage + PlantDoc	ConvNeXt-Tiny	19	224	22,372
C: more field data	PlantVillage + PlantDoc + PlantWild + rice + wheat	ConvNeXt-Tiny, DINOv2-Small, EfficientNetV2-B0, MobileNetV3	26	224–288	33,468

Training setup (all experiments). PyTorch with ImageNet-pretrained backbones from timm, trained on a Kaggle T4 GPU with mixed precision. One epoch trains only the new head, then the whole network is fine-tuned with AdamW, a 10x smaller learning rate for the backbone, cosine decay with warm-up, label smoothing 0.1 and gradient clipping. Training keeps the epoch with the best validation macro-F1 and stops early after 4 epochs without improvement.

Augmentation for field conditions. Random resized crops (35–100% of the photo), flips, rotation, colour jitter, blur, JPEG compression and random erasing imitate field photos: off-centre leaves, uneven light, motion blur and low-quality phone images.

Source-balanced sampling (C). Each class is drawn equally often. Within a class, each dataset's share grows with the square root of its size, so 80 PlantDoc photos of a disease are not drowned out by 1,500 PlantVillage photos of it, without repeating the small set excessively.

Model selection and calibration data are kept apart. Experiment C chooses its best epoch on PlantVillage and PlantWild validation photos; calibration (Section 7) uses PlantDoc and PlantWild validation photos; every reported number comes from test splits that neither step saw. PlantWild validation is used by both selection and calibration, a small overlap noted in Section 12.

Deployment. The server runs D-ConvNeXt-Tiny (30 classes incl. four pest classes, 288 px), which replaced C-ConvNeXt-Tiny after matching its field accuracy. The phone runs C-MobileNetV3 (256 px) exported to ONNX for offline use.

7. Reliability: knowing when it does not know

Every answer carries one of four statuses, decided from calibrated probabilities; on field photos the true disease is inside the model's answer set 90.0% of the time, as targeted.

Temperature scaling. A single temperature T is fitted on calibration photos by minimising negative log-likelihood, so that a stated 80% means right about 80% of the time. For the deployed model T = 0.89, fitted on 684 field photos (PlantDoc and PlantWild validation). Predictions average the logits of each photo and its mirror image (test-time augmentation).

Conformal prediction. With calibration scores s = 1 − p(true class) over n photos and a miss rate α = 0.1, the threshold and the answer set are:

```
\hat{q} = \mathrm{Quantile}\left(s_1,\dots,s_n;\ \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right), \qquad C(x) = \{\, k : p_k(x) \ge 1 - \hat{q} \,\}
```

For the deployed model q̂ = 0.950, so any class with probability of at least 5% stays in the set.

Status	Rule	What the farmer sees
Confident	One class in the set and its probability is at least 50%	Diagnosis, severity and treatment
Ambiguous	2–3 classes in the set	"Could be A or B", a request for a closer photo, no spraying advice
Unknown	Empty set, more than 3 classes, or one class under 50%	"Cannot identify reliably", Kisan Call Centre and KVK contacts
Rejected	Every photo fails the gate	"Not a supported crop leaf", photo tips

The 50% floor was added after testing: a poorly calibrated model produced a one-class set at 39.5% probability, which would have been shown as confident.

Energy gate. The free energy of the raw logits flags photos unlike anything in training:

```
E(x) = -\log \sum_k e^{z_k(x)}
```

The threshold is the 99th percentile on calibration photos ($E \leq -3.87$ passes), so 99% of genuine field leaves pass. It rejected 6 of 7 synthetic non-plant images (blank, black, noise, sky, skin tone); a printed document was returned as unknown, still with no diagnosis.

A negative result: BioCLIP. We tested BioCLIP, a vision-language model trained on biological images, as a zero-shot "is this a crop leaf?" check. On 684 real field leaves, 10% scored below 0.02 while junk images scored 0.001–0.003, so any useful cut-off rejected about 8% of genuine farmer photos. BioCLIP is therefore optional and off by default.

Calibration data must match deployment. A first calibration that mixed easy rice and wheat photos (85% of 1,739 images) made the model over-confident on hard field photos: coverage fell to 58.6% against the 90% target. Recalibrating on field photos only restored 90.0%.

Crop selection. The farmer can name the crop before scanning; the model then chooses only among that crop's classes (probabilities of other crops set to zero and renormalised). On the 541 field test photos this raised accuracy from 69.3% to 74.7%, coverage from 90.0% to 93.0%, and accuracy when confident from 87.4% to 91.1%. It also removes cross-crop answers such as "Tomato healthy" for a rice leaf.

Lesion cross-check. A maize leaf with insect holes (fall armyworm feeding) was diagnosed "possibly rice brown spot", while the lesion model found only 0.5% diseased area. Diseases marked as lesion-type (spots, blights, rusts, molds, blast) are now downgraded to unknown, with the message "no disease spots found; may be insect damage or a nutrient problem", when lesions cover less than 1% of the leaf. The threshold was chosen on the field test photos:

Threshold (lesion area)	Correct diagnoses wrongly downgraded	Wrong diagnoses caught
1% (used)	0 of 237	1 of 96
2%	1 of 237	2 of 96
3%	6 of 237	3 of 96
5%	20 of 237	7 of 96

The check rarely fires on in-scope photos, because wrong answers there are mostly confusions between two real diseases with lesions; it exists for photos of conditions outside the 30 classes. Viruses, mites and insect-damage classes are excluded because they do not produce lesions.

Insect-damage classes (Experiment D). Four pest classes were added to the taxonomy, knowledge base and app, one or two for each cereal crop: maize insect feeding damage (Corn Leaf Infection dataset, about 4,000 field phone photos, mostly fall armyworm), rice hispa (Rice Leafs dataset), and wheat aphids and wheat mites (Aphid and Mite folders of the wheat dataset already in use). Any pest class with fewer than 100 photos is dropped automatically at data preparation. Advice covers integrated pest management only (hand-picking, pheromone traps, natural enemies, neem, leaf-tip clipping, timely irrigation) and never names an insecticide. Pests of tomato, potato and pepper have no dedicated class for lack of field datasets; for those crops the lesion cross-check returns "no disease spots found, may be insect damage". The wheat dataset's validation photos, used for neither training, model selection nor calibration in Experiment D, serve as the first held-out wheat test. Results of the Experiment D ConvNeXt-Tiny (30 classes, calibrated on the same 684 field validation photos): maize insect damage test 100% (223 photos), wheat held-out 90.2% (1,455 photos, incl. aphids and mites), PlantVillage 98.4%, rice 99.7%. On the same 541 field test photos it scores 68.0% against 69.3% for Experiment C (coverage 89.6% vs 90.0%, right when confident 88.4% vs 87.4%), and 73.2% when the farmer names the crop (right when confident 92.1%). Field accuracy is therefore unchanged within noise while four pest classes are gained, so Experiment D replaces C as the deployed model. The fall armyworm photo that started this work is now diagnosed as maize insect feeding damage (67.5%; 94.3% with crop = maize). The offline MobileNetV3 model is still the Experiment C version without pest classes.

8. Severity estimation and explainability

Severity is the share of the leaf covered by lesions, graded 0–4; the lesion model reaches an IoU of 0.63 on 1,247 PlantSeg validation images.

Lesion model. A U-Net with a ResNet-18 encoder (segmentation-models-pytorch) is trained at 384 px on PlantSeg. Its COCO polygon annotations are rasterised into binary lesion masks, whatever the disease. Loss is binary cross-entropy plus Dice; the best epoch (20) is kept by validation IoU.

Leaf area. The leaf is found with the Excess-Green index (2G − R − B) and Otsu's threshold, cleaned with a morphological close and open; lesion pixels always count as leaf. If less than 5% of the photo is leaf, the whole photo is used and the result is flagged.

```
\text{severity} = 100 \times \frac{\text{lesion pixels}}{\text{leaf pixels}}
```

Grade	Label	Lesion area
0	None	up to 1%

Grade	Label	Lesion area
1	Low	1–10%
2	Moderate	10–25%
3	High	25–50%
4	Severe	over 50%

The cut-offs follow a common 0–4 style scale and should be confirmed with agriculture faculty for each crop. Severity is computed only for diseased results that are confident or ambiguous, and is averaged over the photos of one plant. It drives urgency in the advice (Section 9).

Grad-CAM. For the predicted class, gradients of the class score are taken at the backbone's last feature map, averaged per channel and used to weight the activations. The implementation works for any timm model through its forward_features and forward_head split, including vision transformers, whose patch tokens are reshaped to a grid. The app shows the heat map and the lesion overlay next to the photo. A heat map on the background instead of the lesions tells the user to treat the result with caution, the failure mode reported for lab-trained models in the literature.

9. Advisory, weather risk and Hindi

Advice is template-based and safety-first: only a confident diagnosis gets treatment steps, and the app never names a pesticide or dose that is not from an official source.

Knowledge base. All 30 classes have a curated entry: pathogen, description, symptoms, causes, favourable conditions, management steps, organic options, prevention and sources (CIB&RC, TNAU Agritech, ICAR KVK directory). The chemical-control list is deliberately empty until filled from the CIB&RC registration list or ICAR package of practices, because a wrong dose can harm farmers and crops. It is stored in MySQL and synced from a JSON seed on every start.

Advice by status.

- Confident: urgency, management steps, organic options, prevention, a chemical note (ask the KVK for a CIB&RC-registered product; follow label dose and pre-harvest interval), and sources. Viral diseases are always high urgency with "no chemical cure".
- Ambiguous: retake-photo tips, shared general steps of the possible diseases, and "Do not spray until the disease is confirmed".
- Unknown or rejected: retake tips and the Kisan Call Centre (1800-180-1551) and KVK contacts.

Urgency. Severity grade 0–1 gives low urgency, 2 moderate, 3–4 high; a high weather risk raises it one step.

Weather risk. With the user's permission, the backend fetches 72 hours of hourly temperature and humidity from Open-Meteo (free, no key) and counts the hours inside each disease's favourable window; for example late blight counts hours at 10–25 °C with humidity of at least 90%. 24 or more hours is high risk, 8–23 medium. The rules are approximate and documented for review by agriculture faculty. A weather failure never blocks a diagnosis.

Hindi. The interface is fully translated, with reviewed Hindi names for all 30 classes (for example पछेती झुलसा for late blight and पीला रतुआ for yellow rust). Advice and library text are translated by the AI service itself, with Meta's open NLLB-200 model (distilled, 600M parameters) running on the project's own machine, so no account, API key or per-request cost is needed. A general model gets farming words badly wrong (it turned "whorl" into "horse rider", "caterpillars" into "vultures" and "grubs" into "pigs"), so three safeguards were added after translating the whole knowledge base and reading every line: a glossary of about 90 farming terms put in before translation (गोभ, इल्ली, फफूंदनाशक, ठूंठ), reviewed Hindi for 149 sentences the model got wrong, used instead of the model, and translation one sentence at a time, because the model dropped sentences from longer texts. The backend caches each translated sentence in MySQL, pre-translates the whole library at startup (about 4 minutes on a laptop CPU), and falls back to English with a notice if translation is unavailable. Bhashini, the Government of India platform, is supported as an optional alternative when its keys are configured. Translated text is labelled as machine translation.

Ask about this result. A fifth tab on the result page lets the farmer ask follow-up questions, by tapping one of four suggested questions, typing their own, or speaking it (the browser's built-in speech recognition in Hindi or Indian English; the spoken text appears in the box to check before sending, so a misheard question is never sent on its own), in English or Hindi. Google Gemini (free tier, `gemini-3.5-flash`) writes the answer, but the classifier still makes the diagnosis; the language model only explains it. Answers are kept in bounds in layers, all enforced by the backend: only the scan's owner can ask; questions are limited to 300 characters, 20 a day and 10 per scan; a fixed system instruction gives the model only this scan's diagnosis, the advice already shown and the matching knowledge-base entries, and forbids pesticide names, doses and treatment of an unconfirmed diagnosis; the farmer's text is marked as a question, never as instructions; the model must answer in JSON that says whether the question is in scope, and off-topic questions get a fixed reply; Google's safety filter is on; and a final check replaces any answer that names a pesticide or gives a dose, in English or Hindi. Only text is sent, never photos, and every question and answer is stored for review.

10. Implementation

The system is three services: a React app, a Spring Boot backend that owns all data, and a Python AI service that only classifies.

Part	Technology	Responsibility
Frontend	React 19, Vite, Tailwind CSS, Motion, ONNX Runtime Web	Upload 1–5 photos, results with four statuses, heat maps, advice, history, library, English/Hindi, installable offline app
Backend	Java 21, Spring Boot 4.1, Spring Security (JWT), JPA, MySQL	Accounts, image storage, predictions, disease library, advice, weather risk, Hindi translation cache and startup pre-translation, feedback
AI service	Python, PyTorch, timm, FastAPI	Gate, classification with TTA, calibration, conformal decision, severity, Grad-CAM
Training	Kaggle notebooks (T4 GPU)	Manifests, experiments A–C, calibration, evaluation reports, ONNX export

Main backend endpoints. POST /api/predictions/detect (1–5 photos, optional location and explanations), GET /api/predictions/history and /{id} (with advice), GET /api/predictions/{id}/images/{n}, POST /api/predictions/{id}/feedback, GET /api/weather/risk, GET /api/diseases (search, by crop, by id), GET /api/model/info, and authentication (register, login, password reset by email). Errors always return {success: false, message}. The chosen language travels in the Accept-Language header.

Security. Passwords are hashed with BCrypt; new accounts must verify their email with a 6-digit code (valid 10 minutes, 5 wrong tries, a new code at most once a minute, stored only as an HMAC with the server secret) before they receive a login token; sessions use signed JWTs; reset tokens are stored as SHA-256 hashes with 15-minute expiry. Uploads are checked by file signature (JPEG, PNG, WEBP), not by name, and stored under random names; users can read only their own predictions and photos. Advice is translated on the project's own server, so no farmer data is sent to an outside translation service.

Offline mode. The app is an installable PWA: a service worker caches the app shell (about 1 MB), so it opens without internet. Turning on offline mode downloads the MobileNetV3 model (17 MB) and the ONNX Runtime WebAssembly runtime (14 MB) once into Cache Storage. Offline, the phone runs the same pipeline as the server: TTA, temperature,

energy gate and conformal statuses. Photos are saved in IndexedDB and sent for the full analysis with one tap when the connection returns; the last disease library is kept on the device for offline advice.

Two offline findings.

1. Compression: dynamic int8 quantization cut the phone model's field accuracy from 62.5% to 22.0%, and static per-channel int8 to 51.0%. The float32 model (100% top-1 agreement with PyTorch) is shipped instead.
2. Resizing: the browser's canvas resize shifted probabilities enough to change statuses. Reimplementing PIL's antialiased bilinear resize in JavaScript made browser outputs match Python (93.5%, 86.8% and 91.5% against 93.5%, 86.7% and 91.5% on three field photos, identical statuses).

11. Testing and verification

37 automated tests pass, and three end-to-end browser runs in Microsoft Edge covered the full farmer journey with no console errors, page errors or failed requests.

Suite	Tests	What it covers
AI service (pytest)	18	Taxonomy mapping, manifests, duplicate and near-duplicate detection, training, calibration, conformal coverage, metrics, TTA settings, ONNX export, predictor, API errors
Backend: authentication	5	Register, case-insensitive login, invalid tokens, password change and reset
Backend: predictions	10	Statuses and advice, virus urgency, healthy leaves, multi-photo upload, image access between users, upload limits, AI outage, weather failure, feedback validation, weather risk
Backend: translation	4	Local translation through a fake AI service, startup pre-translation, glossary, reviewed sentences and sentence splitting; Bhashini calls against a fake server in its JSON format; sentence cache, English fallback, unsupported languages
Browser: full flow	1 run, 18 screens	Sign-up, two-photo scan with location, every result tab, rejected photo, history, library, Hindi, phone layout

Suite	Tests	What it covers
Browser: Hindi	1 run	Translation applied, unavailable notice, switching EN → HI → EN mid-session
Browser: offline	1 run	App opens with no network, three on-device diagnoses matching Python, pending queue, sync after reconnecting

The backend tests run against an in-memory H2 database with the AI service and weather API replaced by fakes, so they need neither MySQL nor a GPU. The training pipeline is tested end to end on a small synthetic dataset on CPU in about 15 seconds.

Bugs found by testing and fixed.

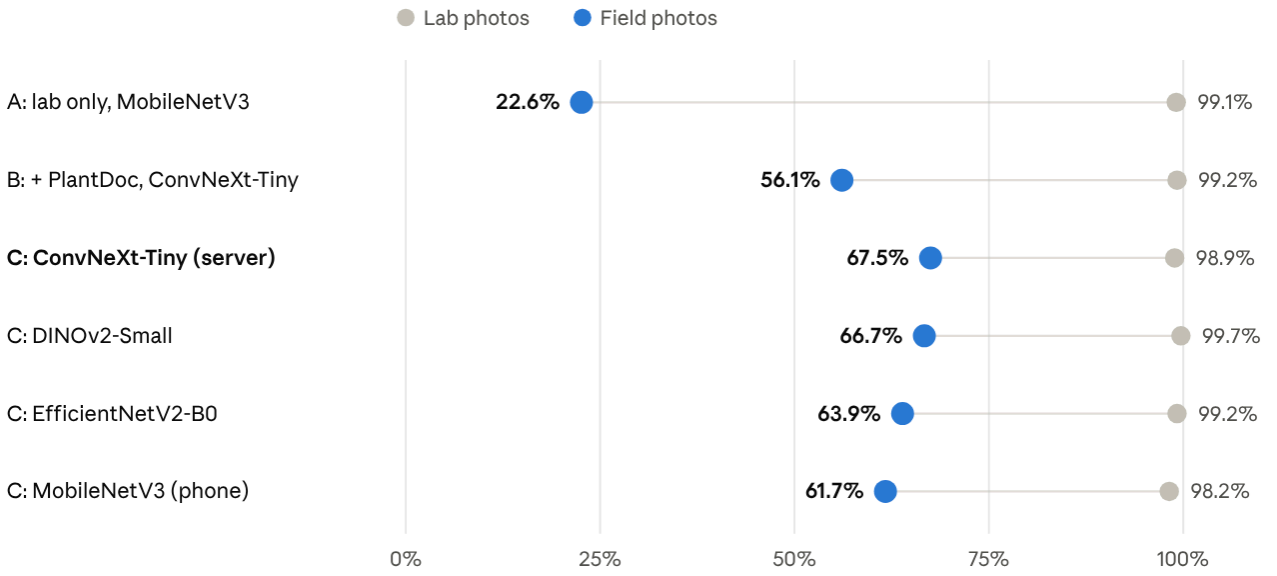
- Training and test splits leaked in the original project (95.12% was measured on images the model had trained on).
- A one-class conformal set at 39.5% probability was shown as confident.
- Calibrating on easy rice and wheat photos made the model over-confident on field photos (coverage 58.6%).
- Dynamic int8 quantization silently destroyed the phone model's accuracy; the old export check used one random-noise image.
- The browser's image resize changed offline statuses.
- Switching language reloaded results with the old language.
- Windows file names: PlantDoc names containing "?" and names differing only in case had to be mapped to rebuild Kaggle's exact splits locally.

12. Results and discussion

On the same 501 field photos, accuracy rose from 22.6% (lab-only model) to 67.5% (deployed model), while lab accuracy stayed between 98.2% and 99.7% for every model.

Field accuracy rose from 23% to 68% while lab accuracy stayed near 99%

Top-1 accuracy, %: 2,662 lab photos (PlantVillage) vs the same 501 field photos (PlantDoc + PlantWild)



measured with ai-service/training/evaluate.py · field set excludes rice and wheat so every model is scored on the same 501 photos

Lab accuracy says almost nothing about field performance: all six models score about 99% in the lab, yet differ by 45 points in the field. Adding PlantDoc alone (B) helped on PlantDoc itself (67.2%) but reached only 52.3% on PlantWild, which it never saw; adding PlantWild, rice and wheat with source-balanced sampling (C) generalised better. Among Experiment C backbones, ConvNeXt-Tiny and DINOv2-Small are within one point of each other; MobileNetV3 trades 6 points for a model small enough for a phone.

Reliability on all 541 field test photos (Experiment C, calibrated on field photos).

Model	True disease in answer set (target 90%)	Shown as confident	Correct when confident
ConvNeXt-Tiny (server)	90.0%	42.5%	87.4%
DINOv2-Small	89.8%	38.3%	84.1%
EfficientNetV2-B0	92.2%	26.8%	89.0%
MobileNetV3 (phone)	91.1%	22.0%	88.2%

The deployed model is right 87.4% of the time when it says "confident", against 67.5–69.3% overall, which is the purpose of the four statuses: the uncertain half of field photos is sent back for a better photo or to an expert instead of being treated. Coverage matches the 90% target for every backbone.

Other components. The severity model reaches an IoU of 0.63 on 1,247 PlantSeg validation images. The offline phone model gives the same top-1 answer as PyTorch on 100% of the 541 field test photos (62.5% accuracy), and browser outputs match Python to within 0.1 percentage point.

What these numbers do not show. The field photos are web photos, not photos from our own region; PlantDoc's 128 test photos give a margin of about ±8 points; and rice (100%) and wheat (no test split) are excluded from the comparison for the reasons in Section 5.

13. Limitations and future work

The largest limitation is the field test data: 541 web photos, no wheat test set, and none of our own local farm photos yet.

Limitations

- PlantDoc has only 128 test photos for our classes, so its accuracy carries a margin of roughly ±8 points; PlantWild (413) is the more reliable field number.
- The rice dataset contains rotated copies of the same photos, so its 100% test accuracy is not meaningful; the wheat dataset shipped no test split.
- Field accuracy (69.3%) is far from lab accuracy (98.9%); the four-status design limits the harm, but about half of field photos end as ambiguous or unknown.
- PlantWild validation photos were used both to choose the best epoch and to calibrate, a small optimistic bias.
- Severity grades, weather rules and disease advice are general and need review by agriculture faculty for each crop and region; pesticide recommendations are intentionally not included.
- Hindi advice is machine translation and needs review by a Hindi-speaking agriculture expert.
- The offline model is weaker than the server model (62.5% vs 69.3% on field photos) and is a 31 MB one-time download.

Future work

1. Collect 30–50 photos per disease from local farms and the KVK in the rabi season (December–February), labelled by an expert, as a local test set.
2. Add a wheat test set and rice photos without augmented copies; add important local diseases such as rice sheath blight, mustard Alternaria blight and wheat stem rust.
3. Use farmer feedback (already stored with each prediction) for active learning: retrain on photos the model got wrong.
4. Distil the server model into the phone model, and try quantization-aware training to shrink the offline download without losing accuracy.

5. Fill the chemical-control table from the CIB&RC list with dose and pre-harvest interval, reviewed by an expert.
6. Add voice input and output in Hindi through Bhashini speech services.

14. Conclusion

A lab-trained classifier that scores 99% can fall to 27% on real field photos, and this project measured that gap rather than reporting the lab number. Adding field photos, balanced sampling and leakage control raised field accuracy to 69.3% across six crops and 26 classes, with lab accuracy unchanged at 98.9%; the deployed Experiment D model adds four pest classes at the same field accuracy (68.0%).

The more important result is behavioural. Calibrated conformal prediction lets the app say "confident", "could be A or B", "cannot tell" or "not a crop leaf", and on field photos the true disease is inside its answer set 90% of the time, as designed. Advice follows the same rule: the app recommends treatment only for confident diagnoses and never invents a pesticide dose.

Around that core, CropCare AI adds severity, visual explanations, multi-photo diagnosis, weather-based risk, Hindi and an offline mode that gives the same decisions on the phone. Several negative findings (BioCLIP's false rejections, int8 quantization losses, calibration on the wrong data) were caught only because every step was measured on field photos; that habit is the project's main lesson.

References

1. Quantifying the reliability gap in cross-domain plant disease classification, Frontiers in Plant Science, 2026.
2. Uncovering bias in the PlantVillage dataset, M. A. Noyan, 2022.
3. Benchmarking in-the-wild multimodal plant disease recognition (PlantWild), T. Wei et al., ACM Multimedia 2024.
4. A large-scale in-the-wild dataset for plant disease segmentation (PlantSeg), Scientific Data, 2025; dataset on Zenodo.
5. PlantDoc dataset, D. Singh et al.
6. PlantVillage dataset on Kaggle.
7. Rice leaf disease images (P. K. Sethy) and wheat disease dataset on Kaggle.
8. BioCLIP 2: emergent properties from scaling hierarchical contrastive learning, 2025.
9. Conformal prediction for uncertainty quantification in crop production decision support, Computers and Electronics in Agriculture, 2025.
10. Plant disease segmentation networks for fast automatic severity estimation, 2025.

11. LeafNet: vision-language benchmark for plant diseases.
12. Evaluating plant disease detection mobile applications: quality and limitations, 2022.
13. Plant disease diagnosis using AI: a case study on Plantix, CGIAR.
14. Bhashini API documentation, Government of India.
15. NLLB Team, "No Language Left Behind: Scaling Human-Centered Machine Translation", Meta AI, 2022; model facebook/nllb-200-distilled-600M.
16. Open-Meteo weather API.
17. Guo et al., "On calibration of modern neural networks" (temperature scaling), ICML 2017; Angelopoulos and Bates, "A gentle introduction to conformal prediction", 2021; Selvaraju et al., "Grad-CAM", ICCV 2017; Liu et al., "Energy-based out-of-distribution detection", NeurIPS 2020.